Curso: Matemática Discreta
Sección: 30
Ivan Morataya — 16667
David Díaz — 25314
David Ibaté — 251732
Miguel Carranza — 24458
José Ceferino — 251043

Proyecto 3: Permutaciones y combinaciones

Guía para entender y utilizar el programa

1. Descripción del problema

Este programa responde dos preguntas: £de cuántas maneras podemos ordenar unos objetos? y £de cuántas maneras podemos elegir algunos de ellos? Además de indicar la cantidad, muestra las posibilidades en una lista.

Los objetos pueden ser letras, números o nombres. Por ejemplo, las letras `a b c d` representan cuatro objetos diferentes. La lista `a a b c` representa cuatro objetos, pero dos son iguales.

El programa se utiliza escribiendo opciones en un menú. No es necesario modificar el código ni saber programar.

£Qué es una permutación?

Es una manera de **ordenar todos los objetos**. El orden sí importa. Con las letras `a b`, los órdenes `(a, b)` y `(b, a)` son dos resultados diferentes.

£Qué es una combinación?

Es una manera de **elegir una cantidad de objetos**. El orden no importa. Elegir `a` y `b` es lo mismo que elegir `b` y `a`; por eso se cuenta una sola vez.

Las cuatro opciones del programa

1. **Permutaciones de objetos diferentes:** ordenar todos los objetos cuando ninguno se repite. Ejemplo: `a b c d`.
2. **Permutaciones de objetos iguales:** ordenar todos los objetos cuando algunos se repiten. Ejemplo: `a a b c`. Intercambiar las dos letras `a` entre sí no crea un orden nuevo.
3. **Combinaciones de objetos diferentes:** elegir algunos objetos de una lista sin repetidos. Ejemplo: elegir 2 de `a b c d`.
4. **Combinaciones de objetos iguales:** elegir algunos objetos de una lista que puede tener repetidos. Ejemplo: elegir 2 de `a a b c`. Solo se pueden usar las copias disponibles.

2. Cómo utilizar el programa

Abrir el menú

Se necesita tener instalado Python 3.10 o posterior y contar con el archivo `proyecto.py`. El programa no requiere instalar otros paquetes.

1. Abre una terminal: es la ventana en la que se escriben comandos. En este equipo puedes abrirla con **Ctrl + Alt + T**.
2. En este equipo, escribe lo siguiente y presiona **Enter**:

   ```
   cd ~/Documentos/MAD/VIS/Proyecto-3-Discreto
   ```

   Este comando abre la carpeta del proyecto. Si guardas el proyecto en otra carpeta, utiliza la ruta de esa carpeta.
3. Escribe este comando y presiona **Enter**:

   ```
   python3 proyecto.py --interactivo
   ```
4. Aparecerá un menú con las opciones 1, 2, 3 y 4 descritas en la página anterior. La opción **5** ejecuta los cuatro ejemplos del proyecto y la opción **0** cierra el programa.

Hacer una consulta, paso a paso

Para elegir dos objetos de la lista `a a b c`:

1. En `Opción:`, escribe `4` y presiona Enter.
2. Cuando solicite los objetos, escribe `a a b c` y presiona Enter. Separa los objetos con espacios, sin comas.
3. En `Cantidad a seleccionar r:`, escribe `2` y presiona Enter. La letra `r` significa cuántos objetos quieres elegir.
4. El programa mostrará `Total: 4` y esta lista:

   ```
   1. (a, a)
   2. (a, b)
   3. (a, c)
   4. (b, c)
   ```
5. El menú vuelve a aparecer. Puedes hacer otra consulta o escribir `0` para salir.

Cómo escribir los datos y leer la respuesta

Cada palabra separada por un espacio cuenta como un objeto. Por ejemplo, `rojo azul verde` son tres objetos. Escribe `a a` para indicar dos copias iguales; `aa` sin espacio representa un solo objeto. El programa distingue `a` de `A`.

`Total` indica cuántas posibilidades existen. Los paréntesis agrupan los objetos de cada resultado. En las opciones 1 y 2 se usan todos los objetos; en las opciones 3 y 4 se pregunta cuántos quieres elegir.

3. Casos de prueba y resultados obtenidos

Cada caso se ejecutó en el programa y se comparó con el cálculo esperado. En las listas de esta página, `abcd` significa el orden `(a, b, c, d)`. Se escribe abreviado para facilitar la lectura.

Caso C1. Ordenar objetos diferentes

Entrada: `a b c d`. **Opción del menú:** 1.

Hay 4 opciones para el primer lugar, 3 para el segundo, 2 para el tercero y 1 para el último. Por tanto, $4 \times 3 \times 2 \times 1 = 24$.

Esperado: 24. Obtenido: 24. Estado: correcto.

```
abcd  abdc  acbd  acdb  adbc  adcb
bacd  badc  bcad  bcda  bdac  bdca
cabd  cadb  cbad  cbda  cdab  cdba
dabc  dacb  dbac  dbca  dcab  dcba
```

Caso C2. Ordenar objetos con repetidos

Entrada: `a a b c`. **Opción del menú:** 2.

Las dos letras `a` son iguales. Si se contaran como diferentes, cada orden aparecería dos veces. Se corrige dividiendo entre 2: $24 \div 2 = 12$.

Esperado: 12. Obtenido: 12. Estado: correcto.

```
aabc  aacb  abac  abca  acab  acba
baac  baca  bcaa  caab  caba  cbaa
```

Caso C3. Elegir objetos diferentes

Entrada: `a b c d`. **Opción:** 3. **Cantidad a elegir:** 2.

Hay $4 \times 3 = 12$ parejas si se cuenta el orden. Como cada pareja aparece en dos órdenes, se divide entre 2: $12 \div 2 = 6$.

Esperado: 6. Obtenido: 6. Estado: correcto.

```
(a, b)  (a, c)  (a, d)  (b, c)  (b, d)  (c, d)
```

Caso C4. Elegir objetos con repetidos

Entrada: `a a b c`. **Opción:** 4. **Cantidad a elegir:** 2.

Esperado: 4. Obtenido: 4. Estado: correcto.

```
(a, a)  (a, b)  (a, c)  (b, c)
```

Se permite `(a, a)` porque hay dos copias de `a`. No se permiten `(b, b)` ni `(c, c)` porque solo hay una copia de cada una. Cambiar el orden de una pareja no crea otra combinación.

4. Pruebas adicionales

También se revisó qué sucede con objetos iguales, selecciones vacías y datos incorrectos. En todos los casos de la tabla, el resultado obtenido coincidió con el esperado. Las opciones son las del menú de la página 2.

Caso	Entrada	Esperado	Obtenido
C5	Opción 2: a a a	1 orden: (a,a,a)	1 orden: (a,a,a)
C6	Opción 4: a a b; elegir 2	2 selecciones: (a,a) y (a,b)	Las mismas 2 selecciones
C7	Opción 3: a b; elegir 0	1 selección vacía: ()	1 selección: ()
C8	Opción 4: lista vacía; elegir 0	1 selección vacía: ()	1 selección: ()
C9	Opción 3: a a b; elegir 2	Error por objetos repetidos	Error por objetos repetidos
C10	Opción 4: a a b; elegir 4	Error: solo hay 3 objetos	Error: r debe estar entre 0 y 3
C11	Opción 4: a a b; elegir -1	Error por cantidad negativa	Error: r debe estar entre 0 y 3
C12	Ordenar a b c d; límite de visualización 2	Total 24; mostrar solo 2	Total 24; 2 visibles y aviso
C13	Opción 2: rojo azul rojo	3 órdenes diferentes	3 órdenes sin repetir

Comprobación del menú

Se ejecutaron las cuatro opciones del menú con los datos de C1 a C4. También se introdujo una cantidad incorrecta y luego una correcta: el programa mostró el error y permitió hacer otra consulta sin cerrarse. Al escribir una palabra en lugar de la cantidad, también informó el error y regresó al menú.

Evidencia de las pruebas

Se ejecutaron diez pruebas automatizadas, que son comprobaciones repetibles del funcionamiento. Todas terminaron correctamente. Dentro de ellas se compararon los resultados con otra forma de enumerarlos para las 364 listas pequeñas de longitud 0 a 5 formadas con a, b y c.

Los registros completos están en `pruebas_automatizadas.txt`, `casos_verificados.txt` y `resultados.txt`, dentro de `documentacion/`. Para repetir las pruebas desde la carpeta principal:

```
python3 -m unittest discover -s tests -v
```

El resultado de la ejecución fue:

```
Ran 10 tests
OK
```

5. Recomendaciones y mensajes del programa

- En las opciones de objetos diferentes, no repitas etiquetas. Si necesitas escribir `a a b c`, utiliza la opción 2 o la 4.
- La cantidad a elegir debe ser un número entero entre 0 y el total de objetos disponibles. No es posible elegir 5 objetos de una lista de 4. Elegir 0 produce una sola selección vacía, escrita como `()`.
- Si aparece `Error`, lee el mensaje y vuelve a ingresar los datos cuando aparezca el menú. Por ejemplo, elige la opción 4 otra vez para corregir el tamaño de una combinación.
- El programa acepta hasta 100 objetos por consulta. Para comenzar, es conveniente utilizar listas pequeñas.
- Por defecto se muestran como máximo 50 resultados. Si hay más, el programa lo indica. El número que aparece en `Total` sigue siendo la cantidad completa. Los cuatro ejemplos de este informe sí se muestran completos.

6. Metodología utilizada

Primero se distinguió si el problema pide ordenar o elegir. Después se revisó si hay objetos repetidos. Con esas dos decisiones se identificaron los cuatro casos que ofrece el menú.

El programa cuenta las posibilidades y genera una lista con cada resultado. Cuando encuentra objetos iguales, lleva el control de cuántas copias quedan disponibles. En las combinaciones evita contar dos veces una misma selección escrita en distinto orden.

Para revisar su funcionamiento se ejecutaron los cuatro ejemplos y se compararon sus listas con los resultados esperados. También se probaron datos incorrectos, listas vacías y otras listas pequeñas. Las diez pruebas automatizadas terminaron correctamente. Los totales obtenidos fueron **24, 12, 6 y 4**, respectivamente.

7. Conclusiones

El programa permite contar y consultar formas de ordenar o elegir objetos sin hacer todas las listas a mano. La decisión principal es saber si el orden importa y si existen objetos repetidos.

Para utilizarlo basta con abrir el menú, elegir una opción e ingresar los datos. En los casos con objetos iguales se respetan las copias disponibles; no se agregan objetos que no estén en la lista inicial.

Referencias

Rosen, K. H. (2019). *Discrete Mathematics and Its Applications* (8.ª ed.). McGraw-Hill Education. Secciones 6.3 (pp. 428–436), 6.5 (pp. 445–456) y 6.6 (pp. 457–460).

Python Software Foundation. *Documentación de Python 3*: módulos itertools, collections, math y unittest. `https://docs.python.org/es/3/`.